

PERIYAR MANIAMMAI
INSTITUTE OF SCIENCE & TECHNOLOGY
(Deemed to be University)
Established Under Sec. 3 of UGC Act, 1956 • NAAC Accredited
think • innovate • transform

CASE STUDY:1

TOPIC: HOSPITAL READMISSION RECORDS-DATA QUALITY & WRANGLING

P24CS103
ADVANCED DATA SCIENCE AND ALGORITHMS

MANIKANDAN K
Reg. No: 126252009
M.Tech CSE

Submitted to,
Dr.M.PRABHA
Assistant Professor CSE

1. ABSTRACT

Data quality plays an important role in healthcare analytics because inaccurate or incomplete medical records can affect the quality of healthcare decisions. This case study focuses on cleaning, preprocessing, and transforming hospital readmission records using Python.

The Diabetes 130-US Hospitals dataset contains approximately 100,000 hospital encounters collected from 130 hospitals in the United States between 1999 and 2008. It includes patient demographics, medical diagnoses, laboratory procedures, medications, admission details, and readmission information.

The main objective is to identify and correct data-quality issues such as missing values, inconsistent categories, duplicate records, and outliers. Python libraries such as Pandas and NumPy are used. Diagnosis codes are also linked with an external ICD-9 lookup table to improve data understanding. The final output is a structured, cleaned, patient-level dataset suitable for further healthcare analysis.

2. INTRODUCTION

Healthcare organizations generate large amounts of patient data every day. These records help hospitals understand patient conditions, treatment patterns, and readmission risks. However, raw healthcare datasets may contain missing values, incorrect entries, inconsistent formats, and duplicate records. These issues reduce the reliability of data analysis.

Data wrangling is the process of converting raw data into a clean, consistent, and meaningful format. This case study uses the Diabetes 130-US Hospitals dataset to perform data profiling, data cleaning, normalization, data integration, and reshaping. The study supports Sustainable Development Goal 3 (SDG 3): Good Health and Well-being.

3. PROBLEM STATEMENT

The raw hospital readmission dataset contains several data-quality challenges:

- Missing values represented by special symbols such as '?'.
- Inconsistent categorical values.
- Potential duplicate patient encounters.
- Extreme values in numerical columns.
- Different data formats and scales.
- Diagnosis information represented using ICD-9 codes.
- Multiple hospital encounters belonging to the same patient.

These issues make direct analysis difficult. Therefore, the dataset must be cleaned, standardized, integrated, and transformed into a patient-level dataset while maintaining the accuracy and meaning of the original medical information.

4. OBJECTIVES

- Understand the structure of the hospital readmission dataset.
- Identify missing values and inconsistent data.
- Detect duplicate records and abnormal numerical values.
- Handle missing values represented by '?'.
- Standardize categorical variables.
- Normalize selected numerical attributes.
- Integrate diagnosis information using an ICD-9 lookup table.
- Transform encounter-level records into patient-level records.
- Document all data-cleaning decisions.
- Prepare a reliable dataset for further healthcare analytics.

5. DATASET DESCRIPTION

Dataset Name: Diabetes 130-US Hospitals for Years 1999–2008

The dataset is obtained from the UCI Machine Learning Repository.

Attribute	Description
Data source	UCI Machine Learning Repository
Number of records	101,766
Number of features	47
Domain	Healthcare
Time period	1999–2008
Hospitals	130
Data type	Categorical and integer
Target variable	Readmission status

Important attributes:

Attribute	Description
encounter_id	Unique hospital encounter ID
patient_nbr	Unique patient ID
race	Patient race category
gender	Patient gender
age	Age group
time_in_hospital	Number of hospital stay days
num_lab_procedures	Number of laboratory procedures

num_medications	Number of medications
diag_1	Primary diagnosis code
diag_2	Secondary diagnosis code
diag_3	Additional diagnosis code
number_diagnoses	Total number of diagnoses
readmitted	Patient readmission status

The target variable 'readmitted' contains three categories:

- <30 – Readmitted within 30 days.
- >30 – Readmitted after 30 days.
- NO – Not readmitted.

6. TOOLS AND TECHNOLOGIES

Tool	Purpose
Python	Main programming language
Pandas	Data cleaning and manipulation
NumPy	Numerical operations
Jupyter Notebook	Interactive coding and analysis
Matplotlib	Data visualization

7. METHODOLOGY

7.1 Data Collection

The Diabetes 130-US Hospitals dataset is downloaded from the UCI Machine Learning Repository. The main files include diabetic_data.csv and IDS_mapping.csv. The CSV files are loaded into Pandas DataFrames.

7.2 Data Profiling

Initial profiling is performed to understand the dataset structure. This includes checking the number of rows and columns, data types, missing values, duplicate records, unique categorical values, and descriptive statistics.

7.3 Handling Missing Values

Special symbols such as '?' are converted into proper missing values (NaN). Missing categorical values may be represented as 'Unknown' where appropriate. Numerical missing values may be handled using median imputation when justified. Columns with substantial missing information are reviewed before deciding whether to retain them. Missing diagnosis codes are not assigned invented diagnoses.

7.4 Handling Outliers

Numerical columns such as time_in_hospital, num_lab_procedures, num_procedures, num_medications, and number_inpatient are examined using the Interquartile Range (IQR) method.

IQR = Q3 − Q1
Lower Limit = Q1 − 1.5 × IQR
Upper Limit = Q3 + 1.5 × IQR

Potential outliers are investigated. Valid extreme medical observations are not automatically deleted.

7.5 Standardizing Categorical Data

Categorical values are standardized by removing unnecessary spaces and ensuring consistent formatting. Race, gender, medical specialty, admission and discharge categories, and readmission status are reviewed. Unknown and invalid categories are preserved or explicitly identified.

7.6 Numerical Normalization

Selected numerical attributes are normalized using Min-Max Scaling. The formula is: Normalized Value = (X − Xmin) / (Xmax − Xmin). The resulting values generally range between 0 and 1.

7.7 ICD-9 Diagnosis Integration

The columns diag_1, diag_2, and diag_3 contain diagnosis codes. A separate ICD-9-CM lookup table is required to map these codes to diagnosis descriptions. The lookup table is merged with the original dataset using the diagnosis code. The supplied IDS_mapping.csv file maps admission type, discharge disposition, and admission source IDs; it is not an ICD-9 diagnosis dictionary.

7.8 Patient-Level Reshaping

The original dataset contains multiple encounters for the same patient. The patient_nbr column is used to group encounters. Patient-level aggregation may include encounter count, average hospital stay, average laboratory procedures, and average medications. Readmission outcomes should be summarized using an explicitly documented rule because the original outcome is encounter-level. The resulting dataset contains one row per patient.

8. IMPLEMENTATION USING PYTHON

The following Python code demonstrates the core data-cleaning and patient-level transformation steps:

```
import pandas as pd
import numpy as np

# Load dataset; convert '?' to missing values
df = pd.read_csv("diabetic_data.csv", na_values=["?"])
```

```
# Inspect dataset
print(df.shape)
print(df.info())
print(df.isnull().sum())

# Check duplicate encounters
print(df.duplicated(subset=["encounter_id"]).sum())

# Handle selected categorical missing values
for col in ["race", "medical_specialty", "payer_code"]:
    df[col] = df[col].fillna("Unknown")

# Standardize text categories
for col in df.select_dtypes(include="object").columns:
    df[col] = df[col].str.strip()

# Convert selected columns to numeric
numeric_cols = [
    "time_in_hospital", "num_lab_procedures",
    "num_procedures", "num_medications", "number_inpatient"
]
for col in numeric_cols:
    df[col] = pd.to_numeric(df[col], errors="coerce")

# Detect potential outliers using IQR
for col in numeric_cols:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr
    print(col, "Lower:", lower, "Upper:", upper)

# Min-Max normalization
for col in numeric_cols:
    minimum = df[col].min()
    maximum = df[col].max()
    if maximum != minimum:
        df[col + "_normalized"] = (
            (df[col] - minimum) / (maximum - minimum)
        )

# Remove duplicate encounter records
df = df.drop_duplicates(subset=["encounter_id"])

# Create one row per patient
patient_df = df.groupby("patient_nbr").agg(
    encounter_count=("encounter_id", "count"),
    average_hospital_stay=("time_in_hospital", "mean"),
    average_lab_procedures=("num_lab_procedures", "mean"),
    average_medications=("num_medications", "mean")
).reset_index()

# Save outputs
df.to_csv("cleaned_hospital_data.csv", index=False)
patient_df.to_csv("patient_level_data.csv", index=False)

print("Data cleaning completed successfully")
```

9. DATA QUALITY ANALYSIS

Quality Dimension	Analysis
Completeness	Identifying missing values
Accuracy	Checking invalid and abnormal entries
Consistency	Standardizing category formats
Uniqueness	Detecting duplicate encounters
Validity	Checking numerical ranges and data types
Integrity	Verifying patient and diagnosis relationships

Every cleaning operation is documented to maintain transparency and reproducibility.

10. RESULTS AND DISCUSSION

The data-wrangling process is designed to produce the following outcomes:

- Missing values are identified and handled using appropriate methods.
- Duplicate encounter records are detected and removed where confirmed.
- Categorical attributes are standardized.
- Numerical attributes are converted into suitable formats.
- Potential outliers are identified using the IQR method.
- Numerical attributes are normalized.
- ICD-9 diagnosis codes are linked to diagnosis descriptions.
- Multiple encounters are consolidated into one patient-level record.

The final dataset is expected to have improved consistency, completeness, and usability. Exact before-and-after missing-value counts, outlier counts, and final row counts must be generated by executing the Python notebook on the downloaded dataset. No unverified statistical results are assumed in this report.

11. SDG 3 – GOOD HEALTH AND WELL-BEING

This case study supports Sustainable Development Goal 3 by improving the quality and usability of healthcare data. Clean and reliable hospital records can help healthcare professionals understand patient histories, analyze readmission patterns, and support better healthcare planning.

- Improved healthcare data management.
- Better understanding of patient readmission patterns.
- More reliable healthcare analytics.
- Evidence-based healthcare decision-making.

12. CONCLUSION

This case study demonstrates the importance of data quality and data wrangling in healthcare analytics. Using the Diabetes 130-US Hospitals dataset, different data-cleaning techniques are applied to identify missing values, inconsistent categories, duplicate encounters, and potential outliers. Numerical normalization, ICD-9 diagnosis integration, and patient-level data transformation help improve the structure and usability of the original dataset.

Python, Pandas, NumPy, and Jupyter Notebook provide an effective environment for performing these operations. The cleaned dataset can be used for future research, exploratory data analysis, and hospital readmission prediction. Thus, this project highlights how data preprocessing can support meaningful healthcare analytics and contribute to SDG 3.

13. REFERENCES

1. Clore, J., Cios, K., DeShazo, J., & Strack, B. (2014). Diabetes 130-US Hospitals for Years 1999–2008. UCI Machine Learning Repository. https://doi.org/10.24432/C5230J
2. UCI Machine Learning Repository. Diabetes 130-US Hospitals Dataset. https://archive.ics.uci.edu/dataset/296/diabetes+130-us-hospitals+for+years+1999-2008
3. Pandas Documentation. https://pandas.pydata.org/docs/
4. NumPy Documentation. https://numpy.org/doc/
5. World Health Organization. Sustainable Development Goal 3 – Good Health and Well-being. https://www.who.int/